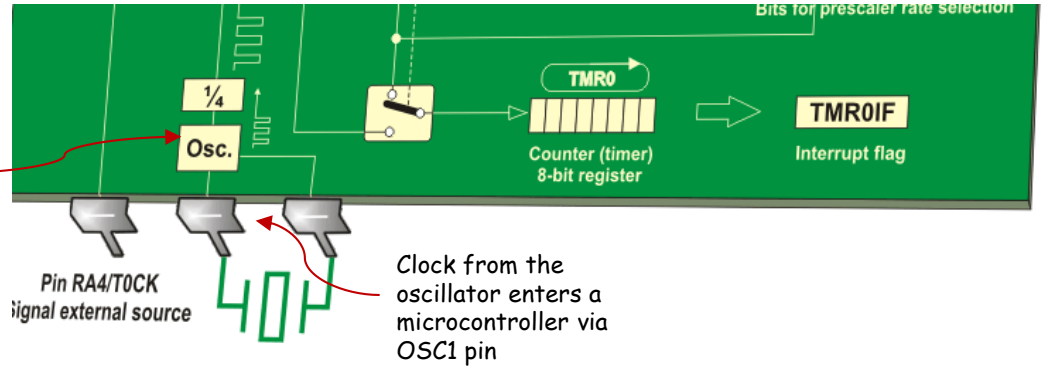
Programming PIC

Clock And Instruction Cycles

Internal circuit divides the clock into four even clocks Q1, Q2, Q3, and Q4 (make up one instruction cycle)



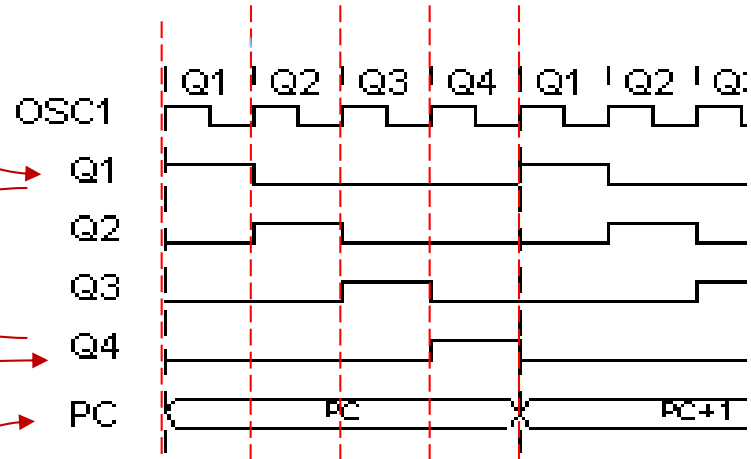
Clock from the oscillator enters a microcontroller via OSC1 pin

Start fetching an instruction every Q1

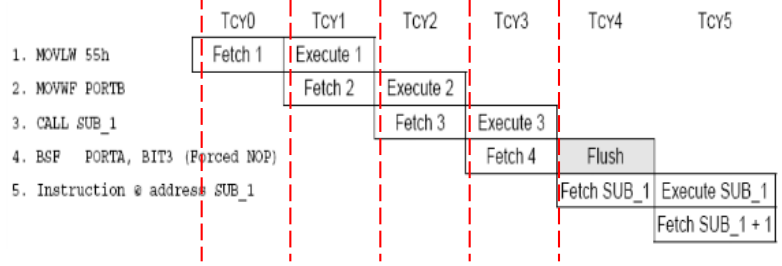
Decoding and execution are done between Q1 and Q4 cycles.

Result stored on Q4

PC holds address of the next instruction to be executed.



Due to pipelining, all instructions execute in a single cycle except branches take two cycles since the fetch instruction is "flushed" from the pipeline while the new instruction is being fetched and then executed.



Instruction Pipeline Flow

Introduction to PIC18 Instruction Set

Mnemonic, Operand		Description	Cycles	Status Affected
Byte-Oriented File Register Operands				
ADDWF	f, d, a	Add WREG and f	1	C, DC, Z, OV, N
ADDWFC	f, d, a	Add WREG and carry bit to f	1	C, DC, Z, OV, N
ANDWF	f, d, a	AND WREG with f	1	Z, N
CLRF	f, a	Clear f	1	Z
COMF	f, d, a	Complement f	1	Z, N
CPFSEQ	f, a	Compare f with WREG, skip =	1 (2 or 3)	None
CPFSGT	f, a	Compare f with WREG, skip >	1 (2 or 3)	None
CPFSLT	f, a	Compare f with WREG, skip <	1 (2 or 3)	None
DECF	f, d, a	Decrement f	1	C, DC, Z, OV, N
DECFSZ	f, d, a	Decrement f, skip if 0	1 (2 or 3)	None
DCFSNZ	f, d, a	Decrement f, skip if not 0	1 (2 or 3)	None
INCF	f, d, a	Increment f	1	C, DC, Z, OV, N
INCFSZ	f, d, a	Increment f, skip if 0	1 (2 or 3)	None
INFSNZ	f, d, a	Increment f, skip if not 0	1 (2 or 3)	None
IORWF	f, d, a	Inclusive OR WREG with f	1	Z, N
MOVF	f, d, a	Move f	1	Z, N
MOVFF	fs, fd	Move fs to fd	2	None
MOVWF	f, a	Move WREG to f	1	None
MULWF	f, a	Multiply WREG with f	1	None
NEGF	f, a	Negate f	1	C, DC, Z, OV, N
RLCF	f, d, a	Rotate left f through carry	1	C, Z, N
RLNCF	f, d, a	Rotate left f (no carry)	1	Z, N
RRCF	f, d, a	Rotate right f through carry	1	C, Z, N
RRNCF	f, d, a	Rotate right f (no carry)	1	Z, N
SETF	f, a	Set f	1	None
SUBFWB	f, d, a	Subtract from WREG with borrow	1	C, DC, Z, OV, N
SUBWF	f, d, a	Subtract WREG from f	1	C, DC, Z, OV, N
SUBWFB	f, d, a	Subtract WREG from f with borrow	1	C, DC, Z, OV, N
SWAPF	f, d, a	Swap nibbles in f	1	None
TSTFSZ	f, a	Test f, skip if 0	1 (2 or 3)	None
XORWF	f, d, a	Exclusive OR WREG with f	1	Z, N
Bit-Oriented File Register Operands				
BCF	f, b, a	Bit clear f	1	None
BSF	f, b, a	Bit set f	1	None
BTFSC	f, b, a	Bit test f, skip if clear	1 (2 or 3)	None
BTFSS	f, b, a	Bit test f, skip if set	1 (2 or 3)	None
BTG	f, d, a	Bit toggle	1	None

Mnemonic, Operand		Description	Cycles	Status Affected
Control Operations				
BC	n	Branch if carry	1 (2)	None
BN	n	Branch if negative	1 (2)	None
BNC	n	Branch if not carry	1 (2)	None
BNN	n	Branch if not negative	1 (2)	None
BNOV	n	Branch if not overflow	1 (2)	None
BNZ	n	Branch if not zero	2	None
BOV	n	Branch if overflow	1 (2)	None
BRA	n	Branch unconditionally	1 (2)	None
BZ	n	Branch if zero	1 (2)	None
CALL	n, s	Call subroutine	2	NOne
CLRWDT		Clear watchdog timer	1	$\overline{TO}$, $\overline{PD}$
DAW		Decimal adjust WREG	1	C
GOTO	n	Goto	2	None
NOP		No operation	1	None
NOP		No operation	1	None
POP		Pop top of return address	1	None
PUSH		Push top of return address	1	None
RCALL	n	Relative call	2	None
RESET		Software device RESET	1	All
RETFIE	s	Return from interrupt enable	2	GIE/GIEH,PEIE
RETLW	k	Return with literal in WREG	2	None
RETURN	s	Return from subroutine	2	None
SLEEP		Go into standby mode	1	$\overline{TO}$, $\overline{PD}$
Literal Operations				
ADDLW	k	Add literal and WREG	1	C, DC, Z, OV, N
ANDLW	k	AND literal with WREG	1	Z, N
IORLW	k	Inclusive OR literal with WREG	1	Z, N
LFSR	f, k	Move literal (12-bit) 2nd word to FSRx 1st word	2	None
MOVLB	k	Move literal to BSR<3:0>	1	None
MOVLW	k	Move literal to WREG	1	None
MULLW	k	Multiply literal with WREG	1	None
RETLW	k	Return with literal in WREG	2	None
SUBLW	k	Subtract WREG from literal	1	C, DC, Z, OV, N
XORLW	k	Exclusive OR literal with WREG	1	Z, N
Data memory to and from Program memory Operations				
TBLRD*		Table read	2	None
TBLRD*+		Table read with post-increment		None
TBLRD*-		Table read with post-decrement		None
TBLRD+*		Table read with pre-increment		None
TBLWT*		Table write	2 (5)	None
TBLWT*+		Table write with post-increment		None
TBLWT*-		Table write with post-decrement		None
TBLWT+*		Table write with pre-increment		None

The WREG Register

D7	D6	D5	D4	D3	D2	D1	D0

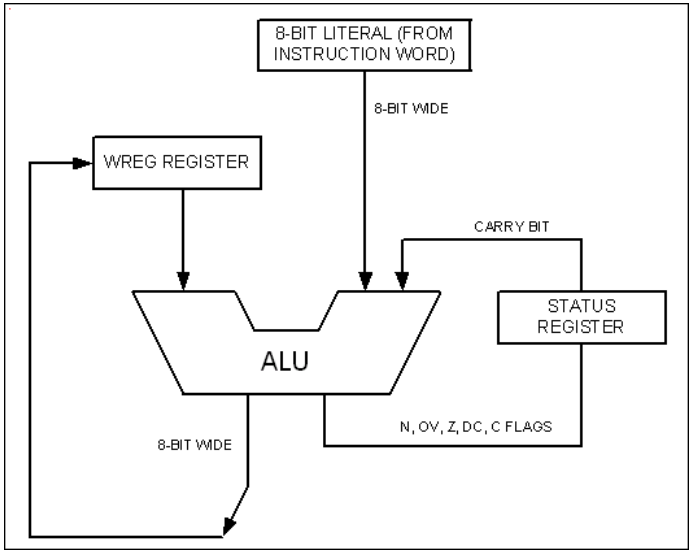
- The WREG (Working Register) Register is one of the most widely used registers of the PIC.
- W, the working register, is used by many.
- Doesn't reside in Data Memory like SFRs
- 8-bit register any data larger than 8 bits must be broken into 8-bits chunks before it is processed.

MOVLW k

("L" for "literal") Working register

MOVLW k ← Load W with the value k

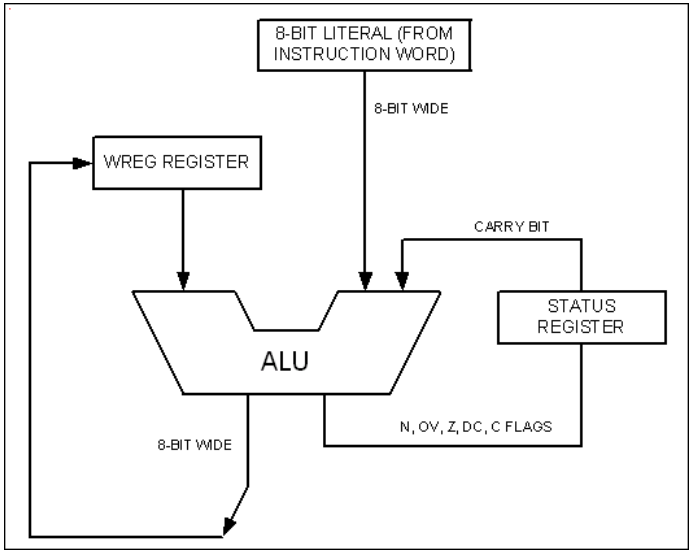
- Moves 8-bit data into WREG (00H ≤ k ≤ FFH)
- Move literal value k into WREG (WREG = k)



Example:

- MOVLW 25H → WREG = 25H
- MOVLW 5AH → WREG = 5AH

ADDLW



ADDLW k;

Add literal value k to WREG (k +WREG)

Example:

MOVLW 12H

ADDLW 16H

0	0	0	1	0	0	1	0
0	0	1	0	1	0	0	0

The WREG Register

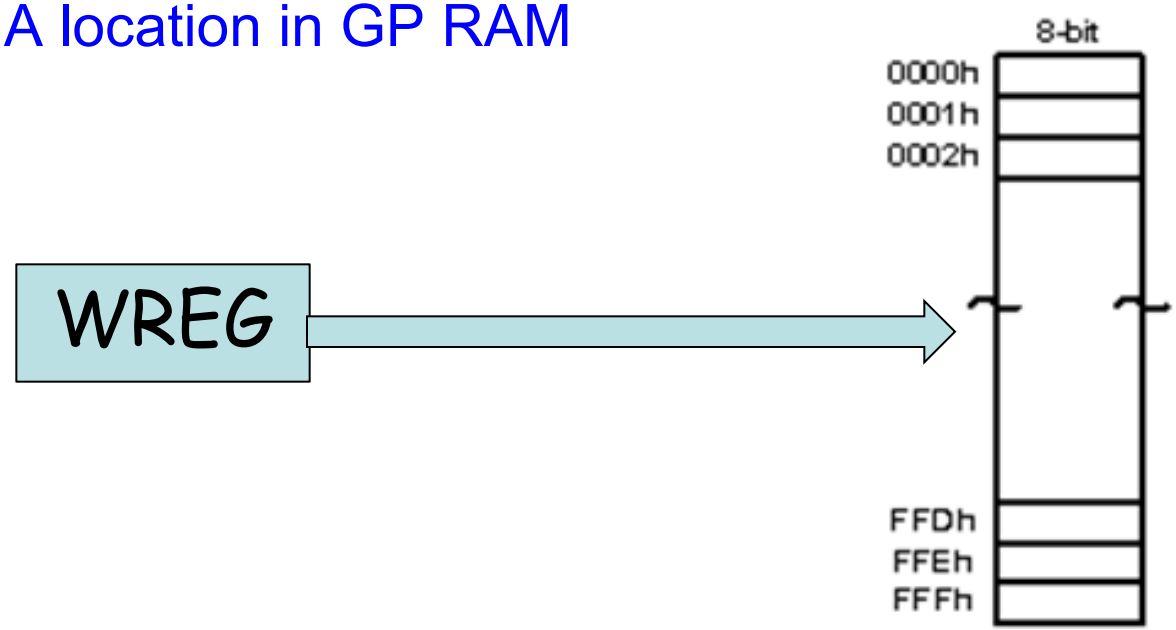
- MOVLW 7F2H; Illegal , becomes F2H
- MOVLW 456H ; Illegal, becomes 56H
- MOVLW 60A5H; Illegal, becomes A5H

MOVWF instruction

- F indicates for a file register

MOVWF Address

- It tells the CPU to copy the source register, WREG, to a destination in the file register.
  - ✓ A location in the SPR
  - ✓ A location in GP RAM



Example:

WRFG

- MOVLW 99H
- MOVWF 12H
- MOVLW 85H
- MOVWF 13H
- MOVLW 3FH
- MOVWF 14H
- MOVLW 63H
- MOVWF 15H
- MOVLW 12H
- MOVWF 16H

99

85

3F

63

12

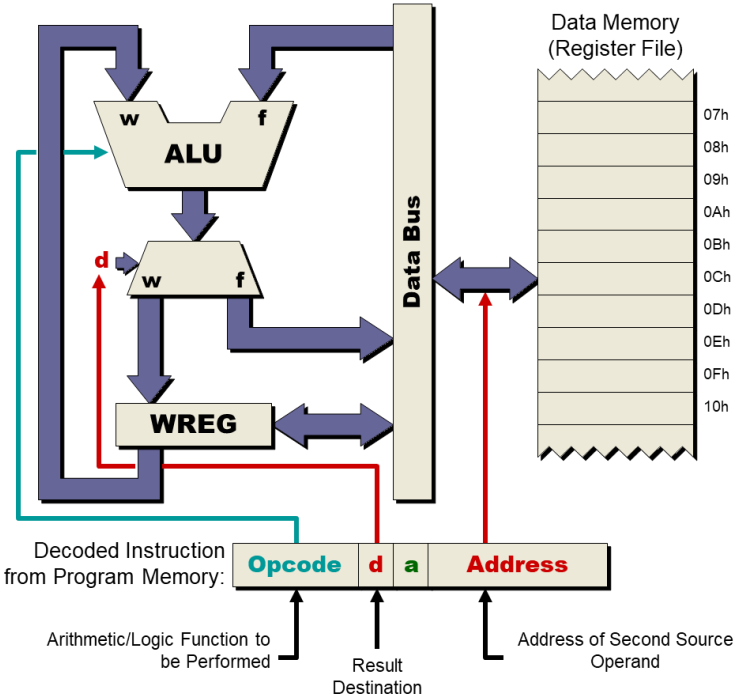
Address	Data
012H	99
013H	85
014H	3F
015H	63
016H	12

Note

- ✓ We cannot move literal values directly into the general purpose RAM location in the PIC18.
- ✓ They must be moved there via WREG.

ADDWF

- ✓ Adds together the content of WREG and a file register location
- ✓ ADDWF File Reg. Address, D
- ✓ The result will be placed in either the WREG or in the file register location
- ✓ D indicates the destination bit
- ✓ If D=0 or (D=w) . The result will be placed in the WREG
- ✓ If D=1 or (D=f) . The result will be placed in the file register



Example

- State the content of file register location and WREG after the following program

MOVLW 0	0
MOVWF 12H	
MOVLW 22H	22
ADDWF 12H, F	
ADDWF 12H, W	44
ADDWF 12H, W	66
ADDWF 12H, W	88

Address	Data
012H	02
013H	
014H	
015H	
016H	

COMF instruction

COMF File Reg. Address, D

- It tells the CPU to complement the content of fileReg and places the results in WREG or in fileReg.

Example

Port B address is F81

- Write a simple program to toggle the SFR of Port B continuously forever.

Solution

```
    MOVLW 55H
    MOVWF PORTB
B1  COMF  PORTB, F
    GOTO  B1
```

55

Address	Data
F81H	~~55H~~ AAH
F82H	
F83H	

DECF instruction

DECF File Reg. Address, D

- It tells the CPU to decrement the content of fileReg and places the results in WREG or in fileReg.
- Example:
  - MOVLW 3
  - MOVWF 12H
  - DECF 12H, F

3

Address	Data
012H	3
013H	
014H	
015H	
016H	

DECF instruction

DECF File Reg. Address, D

- It tells the CPU to decrement the content of fileReg and places the results in WREG or in fileReg.
- Example:
  - MOVLW 3
  - MOVWF 12H
  - DECF 12H, w
  - DECF 12H, w
  - DECF 12H, w

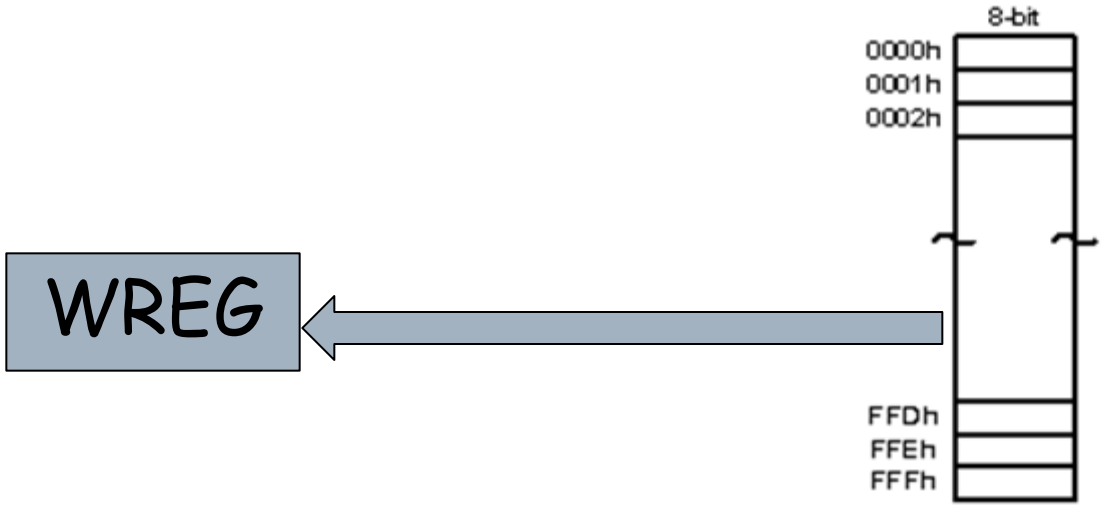
3
2
1
0

Address	Data
012H	3
013H	
014H	
015H	
016H	

MOVF instruction

MOVF File Reg. Address, D

- MOVF File Reg. Address, 0



- If D=1
  - The content of the fileReg is copied to itself.

Example

- Write a simple program to get data from the SFRs of Port B and send it the SFRs of PORT C continuously.

Port B address is F81
Port C address is F82

XX

Solution

```
AGAIN     MOVF  PORTB, W
          MOVWF PORTC
          GOTO  AGAIN
```

Address	Data
F81H	XX
F82H	XX
F83H	

Example

- Write a simple program to get data from the SFRs of Port B Add the value 5 to it and send it to SFRs of PORT C

Port B address is F81
Port C address is F82

- Solution

```
MOVF  PORTB,W
ADDLW 05H
MOVWF PORTC
```

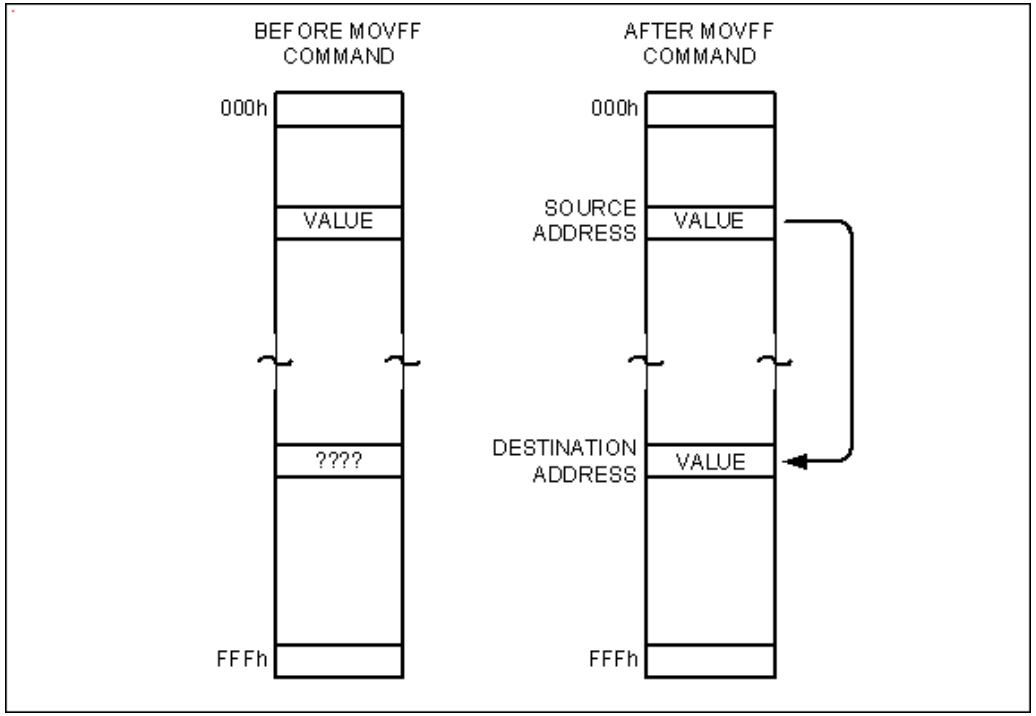
5A

Address	Data
F81H	55H
F82H	5AH
F83H	

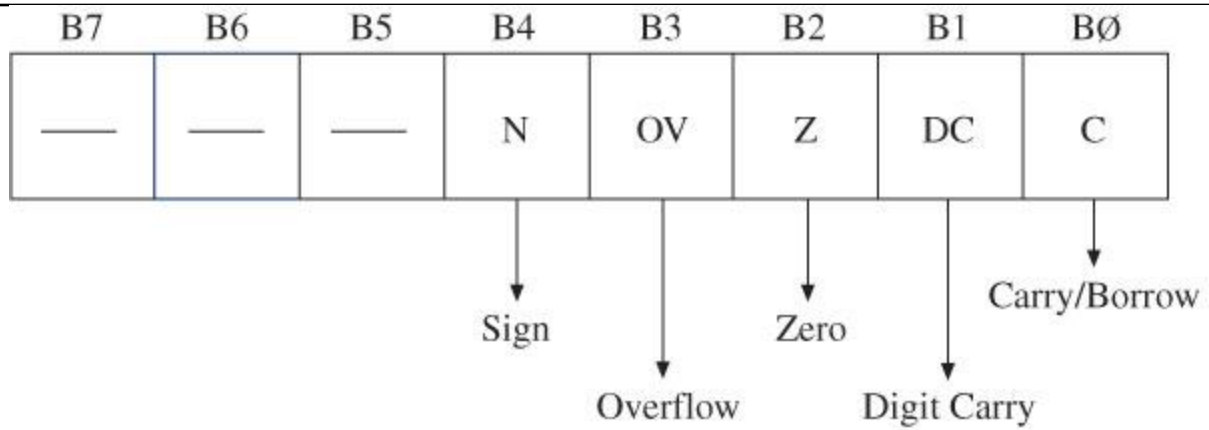
MOVFF instruction

- It copies data from one location in FileReg to another location in FileReg.

  MOVFF Source FileReg, destination FileReg



Flags in Status Register



- C (Carry/Borrow Flag): set when an addition generates a carry and a subtraction generates a borrow
- DC (Digit Carry Flag): also called Half Carry flag; set when carry generated from Bit3 to Bit4 in an arithmetic operation
- Z (Zero Flag): set when result of an operation is zero
- OV (Overflow Flag): set when result of an operation of signed numbers goes beyond seven bits
- N (Negative Flag): set when bit B7 is one of the result of an arithmetic /logic operation

PIC Data Format and Directives

- There is one data type
  - 8 bits
  - It is the job of the programmer to break down data larger 8 bits
- Data type can be positive or negative
- Data format are
  - Hex (default in PIC) 12 or 0x12 or H'12' or 12H
  - Binary B'00010010'
  - Decimal .12 or D'12'
  - ASCII A'c' or a'c'

Assembler Directives

- The Instruction tell the CPU what to do , while Directives give direction to assembler
- Directives : EQU,SET, ORG (Origin), END, LIST
- **EQU (equate)**
  - It is used to define a constant value of a fixed address.
  - It associates a constant number with a data or an address label so that when the label appears in the program , its constant will be substituted for the label .

```
COUNT EQU    0x25
              ...              ....
      MOVLW          COUNT ;WREG = 25H
```

Assembler Directives

- SET
  - Its used to define a constant value or a fixed, it’s the identical to EQU , but the SET may de reassigned later.
- ORG (origin)
  - It is used to indicate the beginning of the address.
- END
  - This indicates to assembler the end of the source (asm)file.

Assembler Directives

- LIST
  - It indicate to the assembler the specific PIC chip for which the program should be assembled

```
LIST P=18F458
```

Structure of Assembly Language

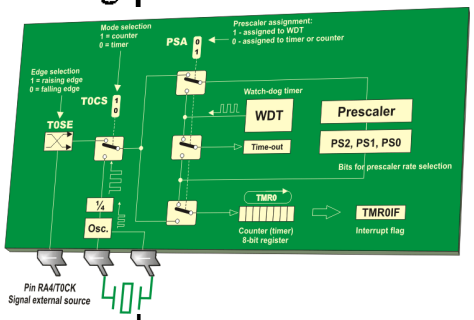
- Series of lines
  - Instruction
  - Directives
- Consists of four field

[label] mnemonic [operands] [;commands]

- Label: refer to line by code (certain length)
- Mnemonic and operands are task that should be executed.
  - Directive don’t generate any machine code and used by assembler

Example

For a PIC18 system of 4 MHz, find how long it takes to execute each of the following instructions:




```
(a) MOVLW        (b) DECF        (c) MOVWF
(d) ADDLW        (e) NOP         (f) GOTO
(g) CALL         (h) BNZ
```

Solution:

The machine cycle for a system of 4 MHz is 1 µs, as shown in Example Appendix A shows instruction cycles for each of the above instructions. Therefore, we have:

Instruction	*Instruction cycles*	*Time to execute*
(a) MOVLW 0x55	1	1 × 1 µs = 1 µs
(b) DECF MYREG	1	1 × 1 µs = 1 µs
(c) MOVWF	1	1 × 1 µs = 1 µs
(d) ADDLW	1	1 × 1 µs = 1 µs
(e) NOP	1	1 × 1 µs = 1 µs
(f) GOTO	2	2 × 1 µs = 2 µs
(g) CALL	2	2 × 1 µs = 2 µs
(h) BNZ	2/1	(2 µs taken, 1 µs if it falls through)

Example

Find the size of the delay in the following program if the crystal frequency is 4 MHz:

```
MYREG EQU   0x08                ;use location 08 as counter

          ORG       0
BACK      MOVLW     0x55        ;load WREG with 55H
          MOVWF     PORTB       ;send 55H to port B
          CALL      DELAY       ;time delay
          MOVLW     0xAA        ;load WREG with AA (in hex)
          MOVWF     PORTB       ;send AAH to port B
          CALL      DELAY
          GOTO      BACK        ;keep doing this indefinitely

;———— this is the delay subroutine
          ORG       300H        ;put time delay at address 300H
DELAY     MOVLW     0xFA        ;WREG = 250, the counter
          MOVWF     MYREG
AGAIN     NOP                   ;no operation wastes clock cycles
          NOP
          NOP
          DECF      MYREG, F
          BNZ       AGAIN       ;repeat until MYREG becomes 0
          RETURN                ;return to caller
          END                   ;end of asm file
```

Solution:

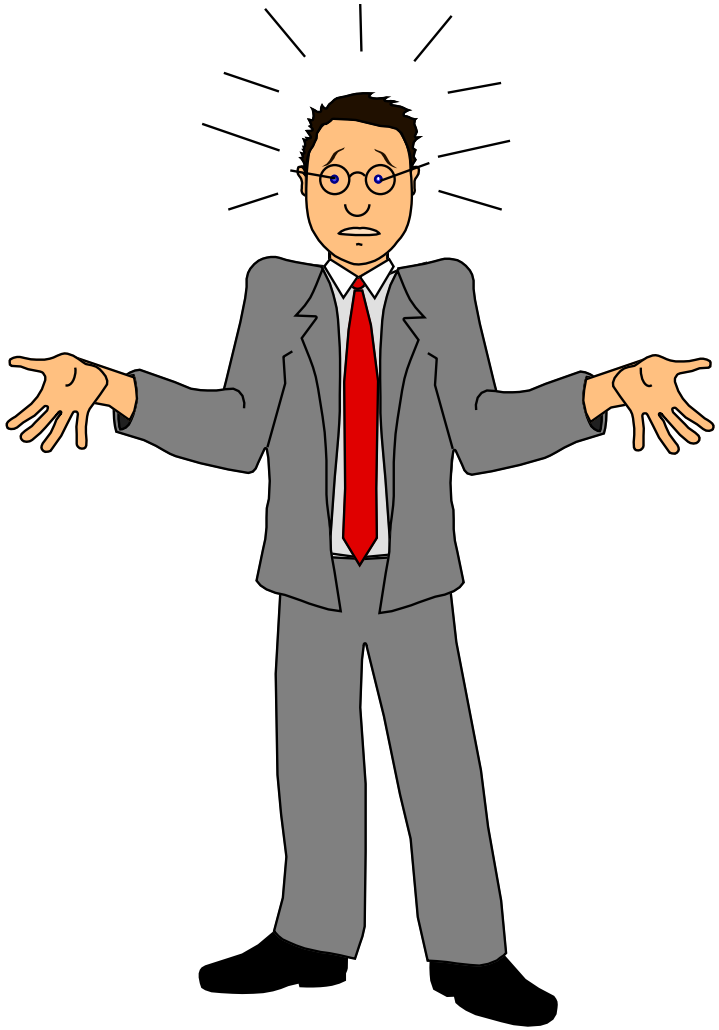
From Appendix A, we have the following machine cycles for each instruction of the DELAY subroutine:

```
                                        Instruction Cycle

DELAY    MOVLW    0xFA                      1
         MOVWF    MYREG                     1

AGAIN    NOP                                1
         NOP                                1
         NOP                                1
         DECF     MYREG,  F                 1
         BNZ      AGAIN                     2

         RETURN                             1
```

Therefore, we have a time delay of $[(250 \times 6) + 1 + 1 + 1] \times 1\ \mu s = 1503\ \mu s$.

Questions